Secure Software Development Lifecycle (SSDLC)

App: Time on the record (Zoom App) · **Repository:** `zoom-meeting-timer`
Owner / security contact: danielbermudez6one9@gmail.com
Last reviewed: 2026-10-17 · **Review cadence:** at least every 12 months, and whenever the process changes

This document describes how security is built into every stage of developing and releasing the App. The App is developed and maintained by a single developer (the "owner"). Where a larger team would separate duties, this document notes the compensating control.

1. Scope

The App is a Node.js / Express server (`server.js`) plus static client pages (`public/`) loaded inside the Zoom client. It:

- completes Zoom OAuth authorization (PKCE) to satisfy the App's required scope. Tokens are exchanged and then **discarded**: they are not stored, logged, or used to call the Zoom API;
- keeps an in-memory, per-meeting recording-time total keyed by meeting UUID;
- exposes a signature-verified deauthorization webhook.

It stores no user accounts, no meeting content, and no personal data beyond the transient meeting UUID (see the Privacy Policy).

2. Lifecycle stages and security activities

2.1 Requirements and design

- **Data minimization by design.** Each feature is checked against "does this need user data?" For example, OAuth tokens are not retained because no feature uses them. Marker text stays in the participant's own panel and is never sent to the server.
- **Zoom platform requirements** are treated as security requirements: OWASP security headers, HTTPS-only endpoints, PKCE OAuth with `state` validation, webhook signature verification, and a deauthorization endpoint.
- **Threat review.** Before a feature that adds an endpoint, stores data, or adds a third party, the owner records the threats considered (spoofing, injection, data exposure, abuse/DoS) in the commit or pull-request description.

2.2 Implementation (secure coding standards)

- No inline scripts or styles. A strict Content-Security-Policy (`script-src 'self'`, no `'unsafe-inline'`) is enforced on every response.
- Untrusted input is never written into HTML. DOM content is built with `textContent`, not `innerHTML` string concatenation.

- Server input is validated against allow-lists (e.g. timer actions).
- Cryptographic randomness comes only from `crypto.randomBytes` (server) and `crypto.getRandomValues` (client).
- Secrets (client secret, webhook secret token) are loaded from environment variables. `.env` is excluded from version control by `.gitignore` and is never committed.
- Third-party code is pulled in only through npm with a committed lockfile. No scripts load from third-party CDNs at runtime (the Zoom Apps SDK is served from the pinned `@zoom/appssdk` package).

2.3 Verification (testing)

Every release must pass the following. Results are saved to `security/evidence/`.

Check	Tool	Command	Pass criteria
SAST	ESLint + `eslint-plugin-security` (plus `no-eval`, `no-implied-eval`, `no-new-func`, `no-script-url`)	`npm run lint:security`	0 errors, 0 warnings
SCA (dependencies)	`npm audit`	`npm audit`	0 high or critical; anything lower is triaged per the Vulnerability Management Procedures
DAST	OWASP ZAP (spider + active scan) against a running instance	see §4	0 high or medium alerts; low/informational alerts triaged and documented
Functional	Manual test in a live Zoom meeting (start / pause / stop cloud recording, reset, deauthorize)	n/a	Timer syncs across participants; no console/CSP errors

`npm run security-check` runs SAST and SCA together.

2.4 Review and release

- All changes are made through git commits with descriptive messages. Before release, the owner reviews the full diff (`git diff <last-release>..HEAD`), with particular attention to the areas in §2.2.
- **Compensating control for a single developer:** automated SAST/SCA/DAST gates (§2.3) must pass, and any change touching authentication, the webhook handler, or security headers gets a second, separate review pass by the owner at least 24 hours after it was written. AI-assisted code review may be used as an additional reviewer.
- A release is not deployed if any §2.3 gate fails, unless a documented risk acceptance (see the Vulnerability Management Procedures) is recorded.

2.5 Deployment and operations

- Production is served over HTTPS only. HSTS is sent on every response.

- Production runs on Render (render.com), which provides HTTPS. Render deploys automatically from the `main` branch of the GitHub repository, so every release is a reviewed, committed version.
- Production secrets are set in Render's environment-variable settings, never in files or the repository.
- Server logs never include tokens, secrets, or request bodies containing personal data.

2.6 Maintenance

- Dependencies are monitored and updated per the Infrastructure and Dependency Management Policy.
- Vulnerabilities, from scans or external reports, are handled per the Vulnerability Management Procedures.
- Security incidents are handled per the Incident Management and Response Policy.

3. Training

The owner keeps current on secure web development (the OWASP Top 10, Zoom App security requirements, Zoom developer announcements) and reviews this document annually.

4. Reproducing the DAST scan

```
PORT=3999 npm start
```

Then, from the OWASP ZAP install directory (ZAP 2.17.0):

```
java -jar zap-2.17.0.jar -cmd -quickurl http://localhost:3999/ -quickout
<repo>/security/evidence/dast-zap-report.html
```

5. Evidence of this process

See security/README.md for the current scan results and the history of findings fixed.
